

Social media case study

Abdullah Esmail

Problem Statement

In today's digital age, social interaction has increasingly shifted toward online platforms. There is a need for a simplified, minimalistic **social media system** that enables users to share content and engage with each other. This system should allow users to create and view posts, interact through likes and comments, and build social connections by following other users.

The goal is to design a system that supports these core interactions while maintaining clear, enforceable rules—such as limiting duplicate likes and managing post content. The system should be scalable, intuitive, and serve as a foundational model for more complex social applications.

Requirements Gathering



System Requirements



Functional Requirements

User Management

- Each user must have a **unique user ID** assigned by the system.
- Each user has name, email, password.
- A user can register a new account
- A user can login to the system
- A user can change their password
- The system should be able to activate or deactivate the user account
- User can view their post, their information

Social Connectivity

- A user can **follow** other users.
- The system should track who follows whom.

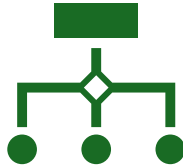
Cont.



Post Creation

A user can **write a post** consisting of **text content**.

A post can optionally include **one image** (only one image allowed per post).



Like Functionality

A user can **“like” a post** created by any user.

A user can **like a specific post only once** (duplicate likes not allowed).



Commenting

A user can **comment** on any post.

Comments are text-only.

A user can **comment multiple times on the same post.**

OOA – Extract the Entities



User



Post



Image



Comment



Follow



like

Attributes of each entity

User Attributes

Attribute	Type	Description
user_id	UUID / INT	Primary key – unique identifier for the user
username	VARCHAR (unique)	Public username or handle
email	VARCHAR (unique)	User's email address for login and communication
password_hash	TEXT	Securely hashed password
first_name	VARCHAR	User's first name
last_name	VARCHAR	User's last name
profile_image_url	TEXT (optional)	URL to the user's profile image
bio	TEXT (optional)	Short biography or personal description
gender	ENUM('Male', 'Female', 'Other')	Gender identity
birthdate	DATE	Date of birth
status	ENUM('Active', 'Inactive', 'Suspended', 'Banned')	Account status
created_at	TIMESTAMP	Timestamp of account creation
updated_at	TIMESTAMP	Timestamp of last profile update

Post attributes

Essential Attributes:

- `id` : Unique identifier
- `user_id` : Reference to the author (`User`)
- `content` : Text content of the post
- `created_at` : When the post was created
- `updated_at` : When the post was last modified
- `visibility` : Public/private/friends (optional based on app needs)

Image

Essential Attributes:

- `id` : Unique identifier
- `url` : Storage path or URL to the image
- `user_id` : Uploader (optional if attached only to posts)
- `post_id` : Associated post (if applicable)
- `alt_text` : Description for accessibility/SEO
- `uploaded_at` : Timestamp

Comment

Essential Attributes:

- `id` : Unique identifier
- `user_id` : Author of the comment
- `post_id` : Post being commented on
- `content` : The comment text
- `parent_comment_id` : (Optional) For threaded/nested comments
- `created_at` : Timestamp
- `updated_at` : Timestamp

Follow

Essential Attributes:

- `follower_id` : User who is following
- `followee_id` : User being followed
- `followed_at` : Timestamp

Like

Essential Attributes:

- `id` : Unique identifier (optional)
- `user_id` : Who liked
- `post_id` : (Or `comment_id` or `image_id`) – What was liked
- `liked_at` : Timestamp

Operations

operations

Operation Category

Authentication

Operation Description

Register a new account

Login

Change password

Delete their account

Profile Management

User can view their info

User can update their info such as name, gender, birth date

User Interactions

User can follow other users

User can view their followers sorted by timestamp

User can view other users who follow them (sorted)

User can search by name for users

Cont.

Posts

Operation Description

View their posts

View details of a specific post

Delete or update their post

Likes & Bookmarks

User can like a post

User can save posts to their bookmark

Comments

User can comment on a post

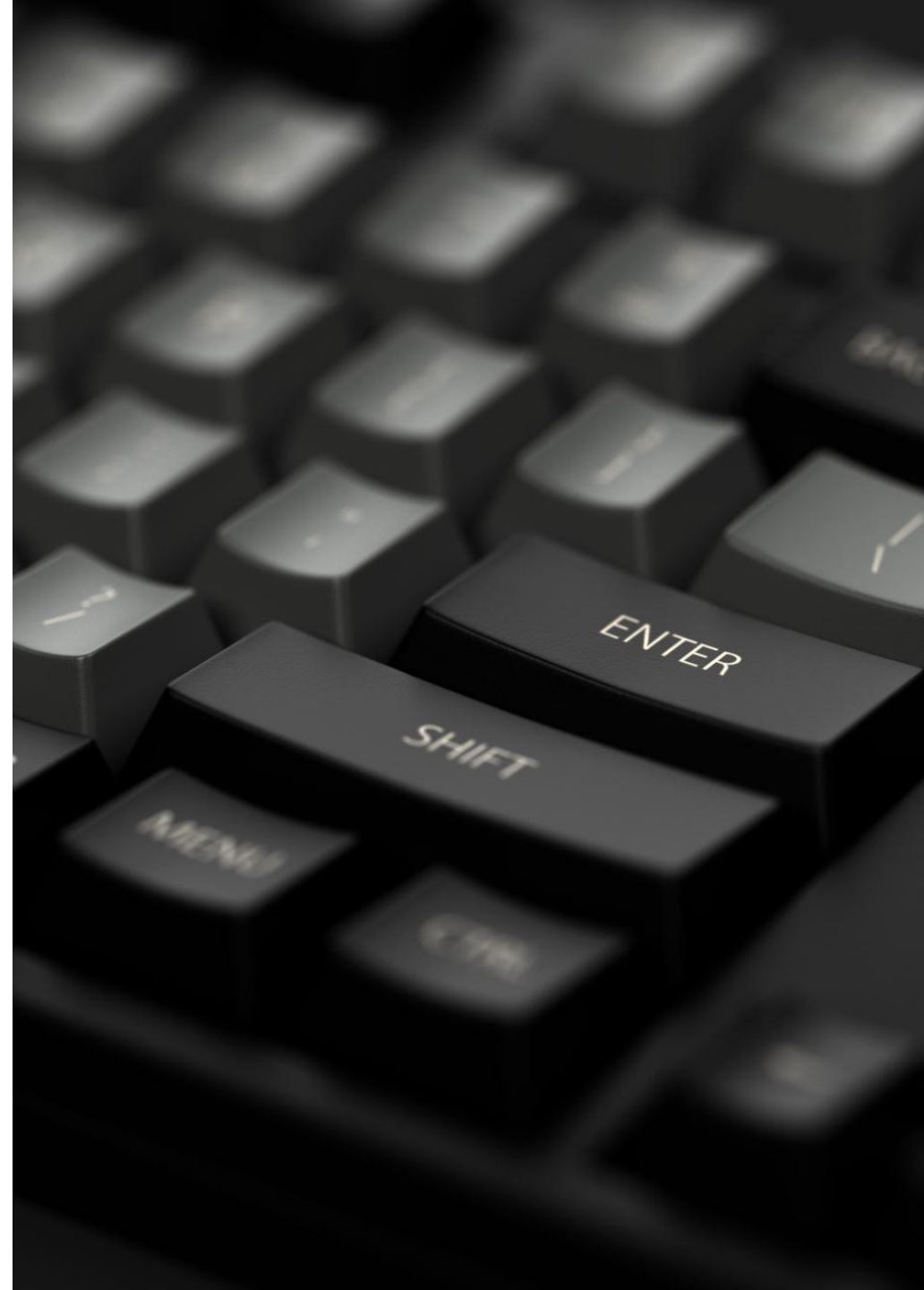
User can reply to a comment

User can edit or delete a comment

Textual Details

Authentication & Account Management

- Register a new account
- Login
- Change password
- ☒ **Logout**
- ☒ **Reset forgotten password (via email)**
- ☒ **Verify email after registration**
- Delete their account



User Profile & Info

User can view their info

User can update their info such as name, gender, birth date

☒ User can upload/update profile image

☒ User can deactivate/reactivate their account

☒ User can update their bio

User Interaction

User can follow other users

User can view their followers sorted by timestamp

User can view other users who follow them sorted by following timestamp

User can search by name for users

☒ **User can unfollow other users**

☒ **User can see who they are following**

☒ **User can block or report another user**

☒ **User can view mutual followers with another user**

Post Operations

View their posts

View details of a specific post

Delete/update their post

☒ Create a new post (text/image)

☒ View others' public posts

☒ Report a post (abusive/inappropriate)

Like, Save & Bookmark

User can like a post

 **User can unlike a post**

User can save posts to their bookmark

 **User can view their saved/bookmarked posts**

 **User can remove a post from bookmarks**

Comment System

User can comment on a post

User can reply on a comment

User can edit or delete a comment

✓ User can like/unlike a comment or reply

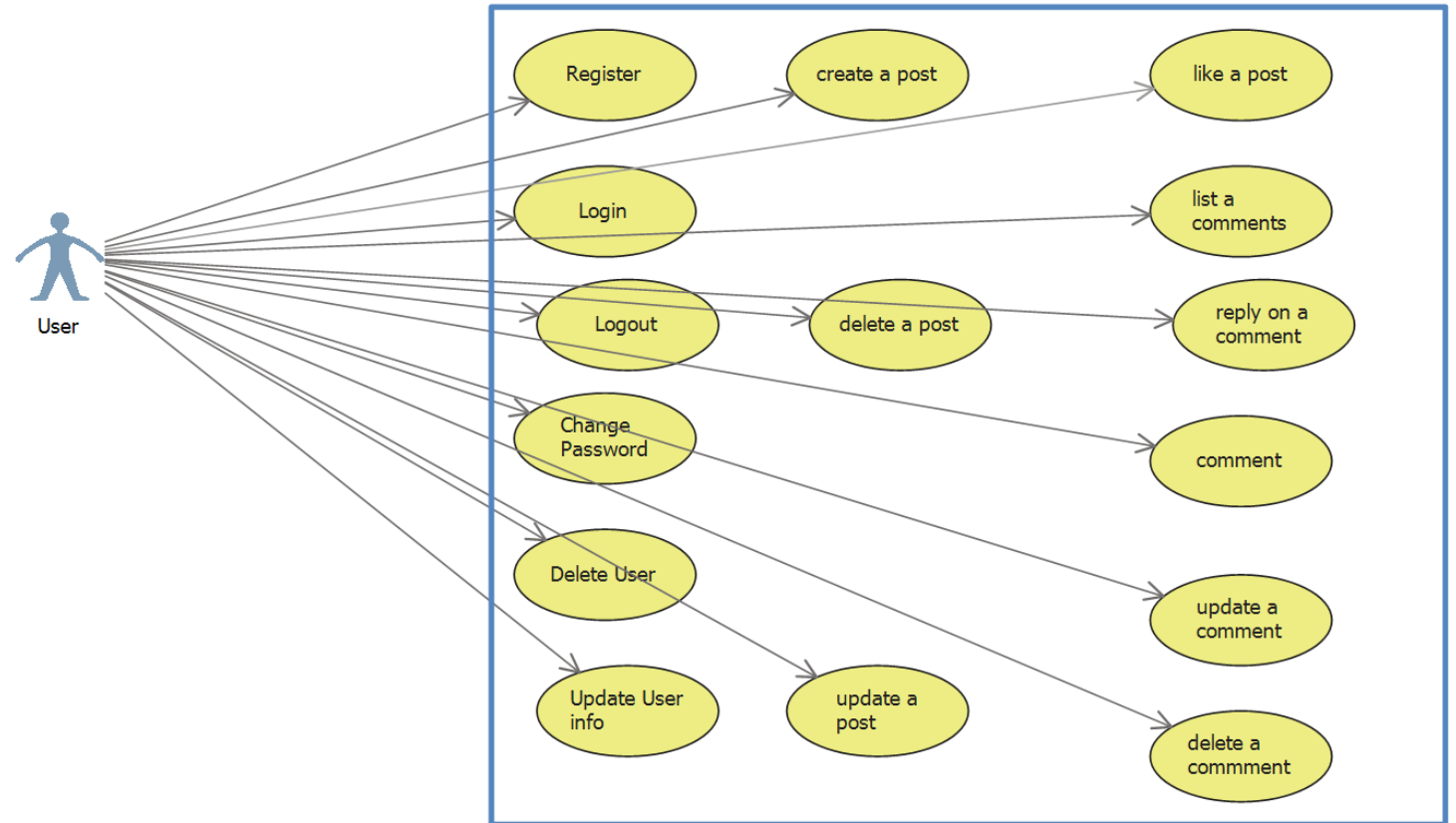
✓ User can report a comment

✓ View all comments on a post (sorted by newest/most liked)

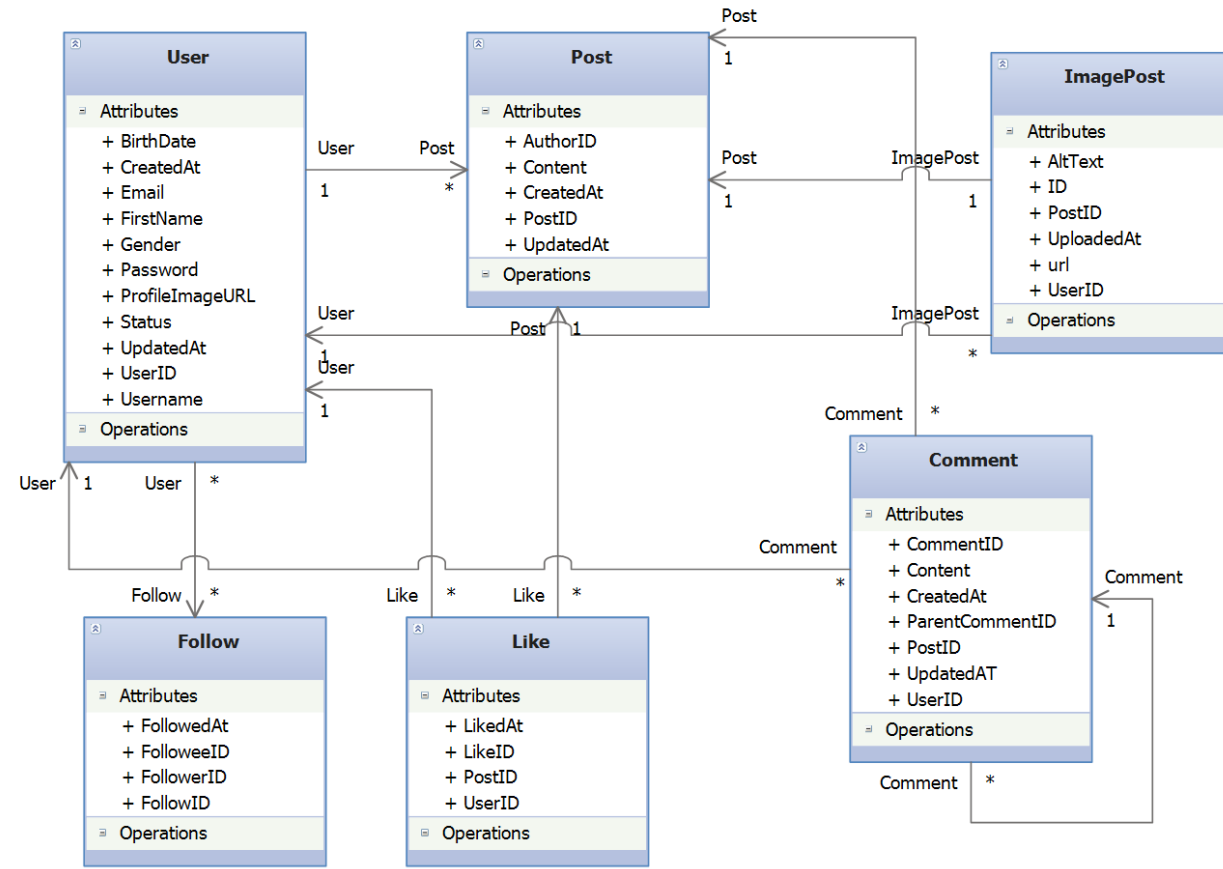
✓ Threaded comment support (see replies to a comment)

Modeling the requirements

Use case diagram



Class diagram

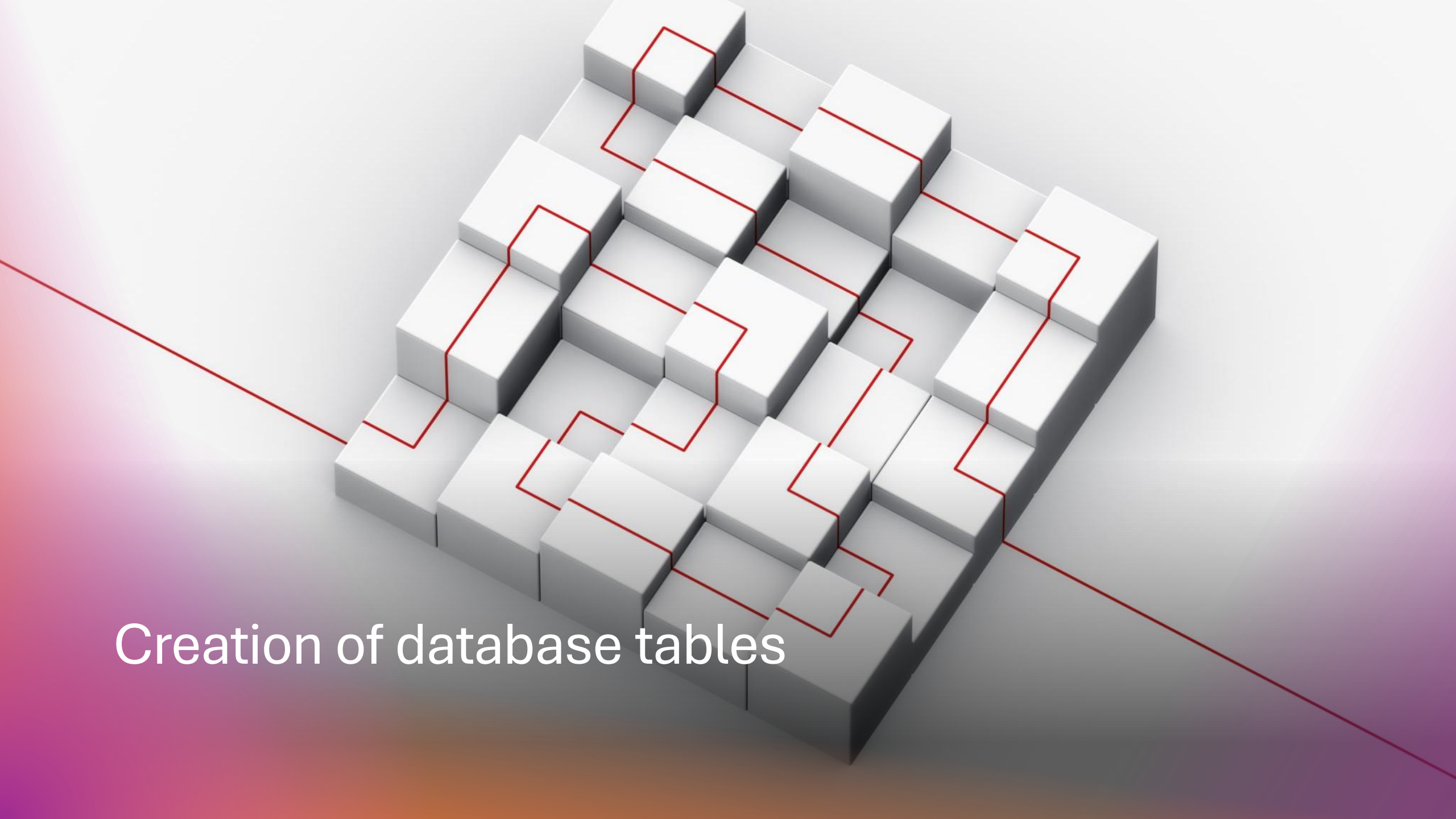


ERD Diagram

- Draw the ERD from the class diagram

Relational Schema or Model

- Draw the Relational Schema and put it here



Creation of database tables

```
CREATE TABLE users (  
    user_id          NUMBER PRIMARY KEY,  
    username         VARCHAR2(50) UNIQUE NOT NULL,  
    email            VARCHAR2(100) UNIQUE NOT NULL,  
    password_hash    VARCHAR2(255) NOT NULL,  
    first_name       VARCHAR2(50),  
    last_name        VARCHAR2(50),  
    profile_image_url VARCHAR2(255),  
    bio              VARCHAR2(1000),  
    gender            VARCHAR2(10) CHECK (gender IN ('Male', 'Female', 'Other')),  
    birthdate        DATE,  
    status            VARCHAR2(20) DEFAULT 'Active' CHECK (status IN ('Active', 'Inactive', 'Suspended', 'Banned')),  
    created_at       DATE DEFAULT SYSDATE,  
    updated_at       DATE DEFAULT SYSDATE  
);
```

```
✓ CREATE TABLE posts (  
    post_id      NUMBER PRIMARY KEY,  
    user_id      NUMBER NOT NULL,  
    content      VARCHAR2(2000) NOT NULL,  
    created_at   DATE DEFAULT SYSDATE,  
    updated_at   DATE DEFAULT SYSDATE,  
    visibility   VARCHAR2(20) DEFAULT 'Public' CHECK (visibility IN ('Public', 'Private', 'Friends')),  
    CONSTRAINT fk_posts_user FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

```
CREATE TABLE images (  
    image_id      NUMBER PRIMARY KEY,  
    url           VARCHAR2(500) NOT NULL,  
    user_id       NUMBER NOT NULL,  
    post_id       NUMBER,  
    alt_text      VARCHAR2(255),  
    uploaded_at   DATE DEFAULT SYSDATE,  
    CONSTRAINT fk_images_user FOREIGN KEY (user_id) REFERENCES users(user_id),  
    CONSTRAINT fk_images_post FOREIGN KEY (post_id) REFERENCES posts(post_id)  
);
```

```
✓ CREATE TABLE comments (  
    comment_id      NUMBER PRIMARY KEY,  
    user_id         NUMBER NOT NULL,  
    post_id         NUMBER NOT NULL,  
    content         VARCHAR2(2000) NOT NULL,  
    parent_comment_id NUMBER,  
    created_at      DATE DEFAULT SYSDATE,  
    updated_at      DATE DEFAULT SYSDATE,  
    CONSTRAINT fk_comments_user FOREIGN KEY (user_id) REFERENCES users(user_id),  
    CONSTRAINT fk_comments_post FOREIGN KEY (post_id) REFERENCES posts(post_id),  
    CONSTRAINT fk_comments_parent FOREIGN KEY (parent_comment_id) REFERENCES comments(comment_id)  
);
```



```
-- FOLLOWS
✓ CREATE TABLE follows (
    follow_id    NUMBER PRIMARY KEY,
    follower_id  NUMBER NOT NULL,
    followee_id  NUMBER NOT NULL,
    followed_at  DATE DEFAULT SYSDATE,
    CONSTRAINT fk_follows_follower FOREIGN KEY (follower_id) REFERENCES users(user_id),
    CONSTRAINT fk_follows_followee FOREIGN KEY (followee_id) REFERENCES users(user_id),
    CONSTRAINT uq_follows UNIQUE (follower_id, followee_id)
);
```



```
CREATE TABLE likes (  
    like_id    NUMBER PRIMARY KEY,  
    user_id    NUMBER NOT NULL,  
    post_id    NUMBER NOT NULL,  
    liked_at   DATE DEFAULT SYSDATE,  
    CONSTRAINT fk_likes_user FOREIGN KEY (user_id) REFERENCES users(user_id),  
    CONSTRAINT fk_likes_post FOREIGN KEY (post_id) REFERENCES posts(post_id),  
    CONSTRAINT uq_likes UNIQUE (user_id, post_id)  
);
```

Procedures and Functions

Be a good Student

register a new user

```
✓ CREATE OR REPLACE PROCEDURE register_user(  
  p_username      IN users.username%TYPE,  
  p_email         IN users.email%TYPE,  
  p_password_hash IN users.password_hash%TYPE,  
  p_first_name    IN users.first_name%TYPE,  
  p_last_name     IN users.last_name%TYPE,  
  p_gender        IN users.gender%TYPE,  
  p_birthdate     IN users.birthdate%TYPE  
) AS  
BEGIN  
  INSERT INTO users (username, email, password_hash, first_name, last_name, gender, birthdate, status, created_at, updated_at)  
  VALUES (p_username, p_email, p_password_hash, p_first_name, p_last_name, p_gender, p_birthdate, 'Active', SYSDATE, SYSDATE);  
END;
```

```
-- Login (returns user ID if valid)
CREATE OR REPLACE FUNCTION login_user(
  p_email          IN users.email%TYPE,
  p_password_hash  IN users.password_hash%TYPE
) RETURN NUMBER AS
  v_user_id NUMBER;
/ BEGIN
  SELECT user_id INTO v_user_id
  FROM users
  WHERE email = p_email AND password_hash = p_password_hash AND status = 'Active';
  RETURN v_user_id;
/ EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
END;
/
```

-- Change password

CREATE OR REPLACE PROCEDURE change_password(

 p_user_id IN users.user_id%TYPE,

 p_new_password IN users.password_hash%TYPE

) AS

BEGIN

 UPDATE users

 SET password_hash = p_new_password, updated_at = SYSDATE

 WHERE user_id = p_user_id;

END;

✓ /



```
-- View user info
CREATE OR REPLACE FUNCTION get_user_info(p_user_id IN users.user_id%TYPE) RETURN SYS_REFCURSOR AS
    rc SYS_REFCURSOR;
✓ BEGIN
    OPEN rc FOR
        SELECT user_id, username, email, first_name, last_name, gender, birthdate, status, profile_image_url, bio
        FROM users WHERE user_id = p_user_id;
    RETURN rc;
END;
✓ /
```



```
-- Follow user
CREATE OR REPLACE PROCEDURE follow_user(
  p_follower_id IN NUMBER,
  p_followee_id IN NUMBER
) AS
BEGIN
  INSERT INTO follows (follower_id, followee_id, followed_at)
  VALUES (p_follower_id, p_followee_id, SYSDATE);
END;
✓ /

-- View followers sorted by followed_at
CREATE OR REPLACE FUNCTION get_followers(p_user_id IN NUMBER) RETURN SYS_REFCURSOR AS
  rc SYS_REFCURSOR;
✓ BEGIN
  OPEN rc FOR
    SELECT u.user_id, u.username, f.followed_at
    FROM users u
    JOIN follows f ON f.follower_id = u.user_id
    WHERE f.followee_id = p_user_id
    ORDER BY f.followed_at DESC;
  RETURN rc;
END;
✓ /
```

```

-- Search users by name
CREATE OR REPLACE FUNCTION search_users_by_name(p_name IN VARCHAR2) RETURN SYS_REFCURSOR AS
    rc SYS_REFCURSOR;
/ BEGIN
    OPEN rc FOR
        SELECT user_id, username, first_name, last_name FROM users
        WHERE LOWER(first_name || ' ' || last_name) LIKE '%' || LOWER(p_name) || '%';
    RETURN rc;
END;
/

-- Update user info
CREATE OR REPLACE PROCEDURE update_user_info(
    p_user_id    IN users.user_id%TYPE,
    p_first_name IN users.first_name%TYPE,
    p_last_name  IN users.last_name%TYPE,
    p_gender     IN users.gender%TYPE,
    p_birthdate  IN users.birthdate%TYPE
) AS
BEGIN
    UPDATE users
    SET first_name = p_first_name,
        last_name = p_last_name,
        gender = p_gender,
        birthdate = p_birthdate,
        updated_at = SYSDATE
    WHERE user_id = p_user_id;
END;
/

```

```
-- Delete user
CREATE OR REPLACE PROCEDURE delete_user(p_user_id IN NUMBER) AS
BEGIN
    DELETE FROM users WHERE user_id = p_user_id;
END;
✓ /

-- View posts for a user
CREATE OR REPLACE FUNCTION get_user_posts(p_user_id IN NUMBER) RETURN SYS_REFCURSOR AS
    rc SYS_REFCURSOR;
✓ BEGIN
    OPEN rc FOR SELECT * FROM posts WHERE user_id = p_user_id ORDER BY created_at DESC;
    RETURN rc;
END;
✓ /

-- Get post details
CREATE OR REPLACE FUNCTION get_post_details(p_post_id IN NUMBER) RETURN SYS_REFCURSOR AS
    rc SYS_REFCURSOR;
✓ BEGIN
    OPEN rc FOR SELECT * FROM posts WHERE post_id = p_post_id;
    RETURN rc;
END;
✓ /
```

```

-- Update post
CREATE OR REPLACE PROCEDURE update_post(p_post_id IN NUMBER, p_content IN VARCHAR2) AS
BEGIN
    UPDATE posts SET content = p_content, updated_at = SYSDATE WHERE post_id = p_post_id;
END;
/

-- Delete post
CREATE OR REPLACE PROCEDURE delete_post(p_post_id IN NUMBER) AS
BEGIN
    DELETE FROM posts WHERE post_id = p_post_id;
END;
/

-- Like a post
CREATE OR REPLACE PROCEDURE like_post(p_user_id IN NUMBER, p_post_id IN NUMBER) AS
BEGIN
    INSERT INTO likes (user_id, post_id, liked_at) VALUES (p_user_id, p_post_id, SYSDATE);
END;
/

```

-- Comment on post or reply to a comment

CREATE OR REPLACE PROCEDURE add_comment(
 p_user_id IN NUMBER,
 p_post_id IN NUMBER,
 p_content IN VARCHAR2,
 p_parent_id IN NUMBER DEFAULT NULL
) AS

BEGIN

INSERT INTO comments (user_id, post_id, content, parent_comment_id, created_at, updated_at)

VALUES (p_user_id, p_post_id, p_content, p_parent_id, SYSDATE, SYSDATE);

END;

✓ /

-- Update comment

CREATE OR REPLACE PROCEDURE update_comment(p_comment_id IN NUMBER, p_content IN VARCHAR2) AS

BEGIN

UPDATE comments SET content = p_content, updated_at = SYSDATE WHERE comment_id = p_comment_id;

END;

✓ /

-- Delete comment

CREATE OR REPLACE PROCEDURE delete_comment(p_comment_id IN NUMBER) AS

BEGIN

DELETE FROM comments WHERE comment_id = p_comment_id;

END;

✓ /

```

-- Compute age as a derived column (use a view)
CREATE OR REPLACE VIEW users_with_age AS
SELECT *, FLOOR(MONTHS_BETWEEN(SYSDATE, birthdate) / 12) AS age
FROM users;

-- Get name of a specific user
✓ CREATE OR REPLACE FUNCTION get_user_name(p_user_id IN NUMBER) RETURN VARCHAR2 IS
    v_name VARCHAR2(200);
✓ BEGIN
    SELECT first_name || ' ' || last_name INTO v_name FROM users WHERE user_id = p_user_id;
    RETURN v_name;
END;
✓ /

-- Get birth date of a specific user
CREATE OR REPLACE FUNCTION get_user_birthdate(p_user_id IN NUMBER) RETURN DATE IS
    v_birthdate DATE;
✓ BEGIN
    SELECT birthdate INTO v_birthdate FROM users WHERE user_id = p_user_id;
    RETURN v_birthdate;
END;
✓ /

-- Get email of a specific user
CREATE OR REPLACE FUNCTION get_user_email(p_user_id IN NUMBER) RETURN VARCHAR2 IS
    v_email VARCHAR2(255);
✓ BEGIN
    SELECT email INTO v_email FROM users WHERE user_id = p_user_id;
    RETURN v_email;
END;

```



-- Get number of users

```
CREATE OR REPLACE FUNCTION count_users RETURN NUMBER IS  
    v_count NUMBER;
```

✓ BEGIN

```
    SELECT COUNT(*) INTO v_count FROM users;
```

```
    RETURN v_count;
```

```
END;
```

✓ /





-- Unfollow a user

CREATE OR REPLACE PROCEDURE unfollow_user(

 p_follower_id IN NUMBER,

 p_followee_id IN NUMBER

) AS

BEGIN

 DELETE FROM follows WHERE follower_id = p_follower_id AND followee_id = p_followee_id;

END;

✓ /





```
-- Get number of followers
```

```
CREATE OR REPLACE FUNCTION count_followers(p_user_id IN NUMBER) RETURN NUMBER IS  
    v_count NUMBER;
```

```
✓ BEGIN
```

```
    SELECT COUNT(*) INTO v_count FROM follows WHERE followee_id = p_user_id;
```

```
    RETURN v_count;
```

```
END;
```

```
✓ /
```



-- Get total number of posts

CREATE OR REPLACE FUNCTION count_all_posts RETURN NUMBER IS
 v_count NUMBER;

✓ BEGIN

 SELECT COUNT(*) INTO v_count FROM posts;

 RETURN v_count;

END;



```
-- Get number of posts of a specific user
CREATE OR REPLACE FUNCTION count_user_posts(p_user_id IN NUMBER) RETURN NUMBER IS
    v_count NUMBER;
/ BEGIN
    SELECT COUNT(*) INTO v_count FROM posts WHERE user_id = p_user_id;
    RETURN v_count;
END;
```





-- Get number of comments on posts by each user

CREATE OR REPLACE VIEW comment_count_per_user AS

SELECT u.user_id, u.username, COUNT(c.comment_id) AS comment_count

FROM users u

JOIN posts p ON u.user_id = p.user_id

JOIN comments c ON p.post_id = c.post_id

GROUP BY u.user_id, u.username;



-- Get number of likes

CREATE OR REPLACE FUNCTION count_all_likes RETURN NUMBER IS
 v_count NUMBER;

 BEGIN

 SELECT COUNT(*) INTO v_count FROM likes;

 RETURN v_count;

 END;

 /

-- Get users who liked a specific post for a specific user

```
CREATE OR REPLACE VIEW users_who_liked_post AS  
SELECT u.user_id, u.username, l.post_id  
FROM likes l  
JOIN users u ON l.user_id = u.user_id;
```

-- Get users who commented on a specific post for a specific user

```
✓ CREATE OR REPLACE VIEW users_who_commented_post AS  
SELECT DISTINCT u.user_id, u.username, c.post_id  
FROM comments c  
JOIN users u ON c.user_id = u.user_id;
```